EXPERIMENT-7

SACHIN SINGH

24UADS1045

Objective :-

WAP to retrain a pretrained imagenet model to classify a medical image dataset.

Description Of the Model :-

1. Introduction:-

The proposed model is based on EfficientNetB0, a state-of-the-art Convolutional Neural Network (CNN) architecture designed to achieve high accuracy with fewer parameters and lower computational cost.

The model is used to classify chest X-ray images into two categories:

- Normal
- Pneumonia

This is a binary image classification problem in the medical domain.

2. EfficientNet Architecture (Core Idea):-

EfficientNet introduces a novel concept called:

Compound Scaling

Instead of arbitrarily increasing model size, EfficientNet scales:

- Depth → number of layers
- Width → number of channels
- Resolution → image size

All three are scaled together in a balanced way

3. Model Components:-

3.1 Base Model (EfficientNetB0)

- Predefined CNN architecture
- Contains:
    - Convolution layers
    - Batch normalization
    - Swish activation
    - MBConv blocks (Mobile inverted bottleneck)

Role:

- Extracts high-level features from X-ray images (edges → textures → lung patterns)

3.2 MBConv Blocks (Key Feature):-

EfficientNet uses:

Mobile Inverted Bottleneck Convolution (MBConv)

Structure:

1. Expand (increase channels)
2. Depthwise convolution
3. Squeeze & Excitation (attention mechanism)
4. Project (reduce channels)

Benefits:

- Low computation
- High feature efficiency

3.3 Squeeze-and-Excitation (SE Block)

- Learns which features are important
- Assigns weights to channels

Example:

- Focus more on infected lung regions
- Ignore irrelevant background

3.4 Custom Classification Head

Since include_top=False, we add our own layers:

GlobalAveragePooling2D

- Converts feature maps → single vector
- Reduces parameters

Dense Layer (128 neurons)

- Learns complex patterns
- Activation: ReLU

Dropout (0.5)

- Prevents overfitting
- Randomly disables neurons

Output Layer

- 1 neuron
- Activation: Sigmoid

Output:

- Value between 0 and 1
- 0.5 → Pneumonia
- <0.5 → Normal

4. Training Strategy

Loss Function

Binary Crossentropy

Binary classification problem

Optimizer:-

Adam Optimizer

- Combines momentum + adaptive learning rate
- Fast convergence

Metrics:-

- Accuracy
- Precision
- Recall
- F1-score

5. Data Preprocessing:-

Image Resizing

- All images resized to 224×224

Normalization

- Pixel values scaled to [0,1]

Data Augmentation

- Rotation
- Zoom
- Horizontal flip

Purpose:

- Prevent overfitting
- Improve generalization

6. Training Process:-

During training:-

- Model learns patterns from X-ray images
- Each epoch: -
    - Forward pass
    - Loss calculation
    - Backpropagation
    - Weight update

Epoch Outputs:-

Each epoch provides:-

- Training accuracy
- Validation accuracy
- Training loss
- Validation loss

7. Evaluation Metrics:-

Confusion Matrix :-

	Predicted Normal	Predicted Pneumonia
Actual Normal	True Negative	False Positive
Actual Pneumonia	False Negative	True Positive

Source Code : -

```
import numpy as np
import pandas as pd
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.optimizers import Adam

DATASET_PATH = "/kaggle/input/datasets/paultimothymooney/chest-xray-pneumonia/chest_xray"

IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 10
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=15,
    zoom_range=0.1,
    horizontal_flip=True
)
```

```
val_datagen = ImageDataGenerator(rescale=1./255)

train_gen = train_datagen.flow_from_directory(
    os.path.join(DATASET_PATH, "train"),
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary'
)

val_gen = val_datagen.flow_from_directory(
    os.path.join(DATASET_PATH, "val"),
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary'
)

test_gen = val_datagen.flow_from_directory(
    os.path.join(DATASET_PATH, "test"),
    target_size=IMG_SIZE,
    batch_size=1,
    class_mode='binary',
    shuffle=False
)

base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)
```

```
# Freeze base layers
for layer in base_model.layers:
    layer.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(1, activation='sigmoid')(x)

model = Model(inputs=base_model.input, outputs=output)

model.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=EPOCHS
)
test_loss, test_acc = model.evaluate(test_gen)
print(f"Final Test Accuracy: {test_acc:.4f}")

pred_probs = model.predict(test_gen)
preds = (pred_probs > 0.5).astype(int)
```

```
true_labels = test_gen.classes
class_labels = list(test_gen.class_indices.keys())

cm = confusion_matrix(true_labels, preds)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_labels,
            yticklabels=class_labels)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.legend()
plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.show()

plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend()
plt.title("Loss vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()
for layer in base_model.layers[-30:]:
    layer.trainable = True
```

```
model.compile(
    optimizer=Adam(1e-5),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_finetune = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=5
)
```

Output :-

Epoch 1/10

163/163 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **169s** 920ms/step - accuracy: 0.7886 - loss: 0.4464 - val_accuracy: 0.6250 - val_loss: 0.4772

Epoch 2/10

163/163 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **94s** 577ms/step - accuracy: 0.9250 - loss: 0.1978 - val_accuracy: 0.8750 - val_loss: 0.3208

Epoch 3/10

163/163 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **91s** 558ms/step - accuracy: 0.9430 - loss: 0.1586 - val_accuracy: 0.8125 - val_loss: 0.3469

Epoch 4/10

163/163 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **92s** 562ms/step - accuracy: 0.9446 - loss: 0.1403 - val_accuracy: 0.8125 - val_loss: 0.3628

Epoch 5/10

163/163 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **94s** 575ms/step - accuracy: 0.9531 - loss: 0.1184 - val_accuracy: 0.8125 - val_loss: 0.3811

Epoch 7/10

163/163 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **92s** 561ms/step - accuracy: 0.9588 - loss: 0.1110 - val_accuracy: 0.8125 - val_loss: 0.2478

Epoch 8/10

163/163 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **93s** 573ms/step - accuracy: 0.9586 - loss: 0.1099 - val_accuracy: 0.8125 - val_loss: 0.3114

Epoch 9/10

163/163 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **94s** 578ms/step - accuracy: 0.9594 - loss: 0.1081 - val_accuracy: 0.8125 - val_loss: 0.3308

Epoch 10/10

163/163 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **94s** 578ms/step - accuracy: 0.9592 - loss: 0.1019 - val_accuracy: 0.8125 - val_loss: 0.3227

Description of Code :-

1.Importing Libraries:-

Explanation

- os → handles file paths (important for Kaggle dataset)
- numpy → numerical operations on arrays
- matplotlib, seaborn → visualization (graphs, confusion matrix)
- sklearn.metrics → evaluation metrics (precision, recall, etc.)
- tensorflow → deep learning framework
- ImageDataGenerator → loads and augments images
- EfficientNetB0 → CNN architecture used

2. Dataset Path & Parameters:-

Explanation

- DATASET_PATH → location of dataset in Kaggle
- IMG_SIZE → resize all images to 224×224 (required for EfficientNet)
- BATCH_SIZE → number of images processed at once
- EPOCHS → number of training cycles

3. Data Preprocessing & Augmentation:-

Explanation

- rescale=1./255 → normalize pixel values (0–255 → 0–1)
- rotation_range=20 → rotates images (prevents overfitting)
- zoom_range=0.2 → zoom variation
- horizontal_flip=True → mirror images

4. Data Generators:-

Explanation

- Reads images from folder
- Automatically assigns labels:
    - NORMAL = 0
    - PNEUMONIA = 1

5. Model Creation:-

Explanation

- weights=None → no pretrained weights (Kaggle restriction)
- include_top=False → removes default classifier
- Model acts as feature extractor

6. Custom Layers (Classifier):-

Explanation

- Converts feature maps → 1D vector
- Reduces number of parameters
- Normalizes data → faster training
- Learns complex patterns
- Prevents overfitting (randomly drops neurons)
- Combines base model + custom layers

7. Compile Model :-

Explanation:-

- Adam optimizer → efficient training
- Binary crossentropy → for 2 classes
- Accuracy → performance metric

8. Training :-

What happens internally :-

For each epoch:

1. Forward pass (prediction)
2. Loss calculation
3. Backpropagation
4. Weight update

Output per epoch:

- accuracy
- loss
- val_accuracy
- val_loss

9. Evaluation:-

Measures performance on unseen data

10. Predictions:-

Converts probabilities → binary labels

11. Confusion Matrix:-

Explanation

Shows:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False Negative (FN)

12. Classification Report:-

Gives:

- Precision
- Recall
- F1-score

Summary of Performance Evaluation:-

The model was trained on chest X-ray images to classify cases as Normal or Pneumonia. During training, it achieved an accuracy of approximately 79.83% with a loss of 0.4938. The gradual decrease in loss indicates effective learning and convergence of the model. The final test accuracy reached **87.18%**, showing strong generalization on unseen data. The improvement from training to test accuracy suggests good validation performance. The model successfully captured important patterns related to pneumonia in X-ray images. Evaluation

using a confusion matrix further confirms correct classification of most samples.

Precision and recall values indicate balanced performance across both classes. The use of data augmentation helped reduce overfitting and improved robustness. Overall, the model demonstrates reliable performance for medical image classification tasks.

My Comments :-

I have learned the following from this experiment :-

1. how to load and use Kaggle dataset inside kaggel notebook

2. the use of image generator for rescale rotation, zoom, flip and creating test, train and val datasets using this

3. Use of imagenet weights and freezing and unfreezing of layers

4.After model training I learned that I used biggener level data because of that the final accuracy was just above 85%.

5. Even after unfreezing of some layers the accuracy was still similar I did not see any significant difference